Hackage: https://hackage.haskell.org/package/free
[Comonad reader] Free monads part 1: https://ekmett.github.io/reader/2011/free-monads-for-less/index.html
part 2: https://ekmett.github.io/reader/2011/free-monads-for-less-2/index.html
part 3: https://ekmett.github.io/reader/2011/free-monads-for-less-3/index.html

Free monad
A **formal representation** of a monad by formal **nesting** of values.
Bind (>>=) just formally puts the new monad inside the last monad i.e. inner most without **collapsing** (i.e. **join)** the monadic layers.
Hence a free bind does NOT produce a monadic **computation (aka interpretation)**.
Hence a free monad allows computation to be **deferred** until the layers are collapsed, giving the programmer control.

```
data Free f a where
  Pure :: a -> Free f a                          a = nested value
  Free :: f (Free f a) -> Free f a               f (Free f a) ~ f (f (f (f ... f a) ...)))

-- if data A a = A a | B a
Pure 1 :: Free A a                               ~ 1
Free (A (Pure 1)) :: Free A a                    ~ A 1
Free (B (Free (A (Pure 1)))) :: Free A a          ~ B (A 1)
Free (A (Free (B ... (Free (A (Pure 1))..) :: Free A a   ~ A (B (... (B (A 1))...)


(>>=) :: m a -> (a -> m b) -> m b                Normal bind
m >>= k                                          m a -> m b

fmap :: (a -> b) -> Free a -> Free b             Free fmap
fmap f (Pure a) = Pure $ f a
fmap f (Free t) = Free $ (f <$>) <$>  t

(>>=) :: Free f a -> (a -> Free f b) -> Free f b  Free bind
Pure a >>= k   = k a
Free t >>= k   = Free ((>>= k) <$> t)

do
  x0 <- liftF (A 1)                              ~ A a1
  x1 <- liftF (B (x0 * 2))                       ~ A (B 2)

  x2 <- liftF (B (x1 * 2))                       ~ A (B (B 4))
  x3 <- liftF (A (x2 * 2))                       ~ A (B (B (A 8)))

foldFree :: Monad m =>                           Collapse and compute
  (forall x. f x -> m x) -> Free f a -> m a
foldFree _ (Pure a) = pure a
foldFree g (Free f) = g f >>= foldFree g         Real join >>=
```

e.g. Maybe vs Free Maybe

```
-- Maybe
Just 1
Just 1 >> Just 2                    -- Just 2
Just 1 >> Just 2 >> Just 3          -- Just 3
```

```
-- Free Maybe
justF :: a -> Free Maybe a
justF = liftF . Just
justF 1                              -- Free (Just (Pure
1))
justF 1 >> justF 2                   -- Free (Just (Free
(Just (Pure 2)))
justF 1 >> justF 2 >> Just 3         -- Free (Just (Free
(Just (Free (Just (Pure 3))))))
```

fold Free Maybe
i.e. compute

```
foldFree :: Monad m =>
  (forall x. f x -> m x) -> Free f a -> m a
foldFree _ (Pure a) = pure a
foldFree g (Free f) = g f >>= foldFree g         --
compute real join


foldFree id $ justF 1                            -- Just
1
foldFree id $ justF 1 >> justF 2                 -- Just
2 i.e. same as Just 1 >> Just 2
foldFree id $ justF 1 >> justF 2 >> Just 3       -- Just
3
```

Church-style Free monad
Codensity
Free monad in **CPS** style
Naive free monad is inefficient because it needs to pass through **all the layers** to add **bind** a new value.
i.e. bind (>>=) contains a **recursive** call on itself.
i.e. same as appending to the **end** of a list.
This can be improved by using a **continuation**.

```
  data F f a where
    F :: forall r. ((a -> r) -> (f r -> r) -> r) -> F f a                (a -> r)       ~
Pure :: a -> Free f a
                                                                         (f r -> r)     ~
Free :: f (Free f a) -> Free f a
  -- r is determined by caller
  F fld = fld Pure Free :: Free f a
  F fld = fld pure join :: f a                                           if f is a monad

  -- e.g. if data A a = A a
  F (\kp _ -> kp 1) :: F A Int                                           ~ Pure 1  i.e. Free
equivalent                              ~ 1
  F (\kp kf -> kf (A (kp 1)) :: F  A Int                                 ~ Free (A (Pure 1))
                                        ~ A 1
  F (\kp kf -> kf (A (kf (A (kp 1 ))) :: F A Int                         ~ Free (A (Free (A
(Pure 1)))                              ~ A (A 1)
  F (\kp kf -> kf (A (kf (A ... (kf (A (kf (A (kp 1))) ... )))           ~ Free (A (Free
```

(A ... (Free (A (Free (Pure 1))) ...))) ~ A (A (A ... (A (A 1)) ...)))

```
-- Efficiency of Free vs F
fmap :: (a -> b) -> Free f a -> Free f a
fmap f (Pure a) = Pure $ f a
fmap f (Free t) = Free $ (f <$>) <$> t                    Recursive fmap.
Expensive

fmap :: (a -> b) -> F f a -> F f b
fmap f (F fld) = F $ \kp kf -> fld (kp . f) kf            No recursion

(>>=) :: Free f a -> (a -> Free f b) -> Free f b
Free t >>= k = Free $ (>>= k) <$> t                       Recursive >>=.
Expensive!

(>>=) :: F f a -> (a -> F f b) -> F f b
F fld >>= k = F $ \kp kf = fld (run kp kf . k) kf         No recursion
  where run kp kf (F x) = x kp kf
```

Free (naive free) vs F (CPS free)
F = **fold** function of Free

```
F :: forall r. ((a -> r) -> (f r -> r) -> r) -> F f a     F constructor
foldFree :: (a -> r) -> (f r -> r) -> Free f a -> r       Free fold

F (\kp _ -> kp 1)                                         ~ fold Pure 1
foldFree kp _ (Pure 1) = kp 1

F (\kp kf -> kf (A (kp1)))                                ~ fold (Free (A (Pure
1))
foldFree kp kf (Free (A (Pure 1))) = kf (A (kp 1)))
```

```
-- F  <=>  Free conversion
fromF :: F f a -> Free f a
fromF (F fld) = fld Pure Free

toF :: Free f a -> F f a
toF (Pure a) = F $ \kp _ -> kp a
toF (Free t) = F $ \kp kf -> kf (run kp kf . toF <$> t)
  where
    run kp kf (F x) = x kp kf
```

e.g. bind and fold with F

```
-- Setup
data A a = A a deriving Functor

liftA :: a -> F A a
liftA = liftF . A
```

Bind and fold with A

```
-- e.g. bind
F $ \kp _ -> kp 1                    -- pure 1 :: Free A
Int
```

```
F $ \kp kf -> kf $ A $                -- liftA 1 :: Free
A Int
               kp 1

F $ \kp kf -> kf (A $                 -- liftA 1 >> liftA
2
               1 & (\_ ->
                kf $ A $
                 kp 2))

F $ \kp kf -> kf (A $                 -- equivalent to
liftA 1 >> liftA 2
               kf $ A $
                kp 2)

F $ \kp kf -> kf (A $                 -- liftA 1 >> liftA
2 >> liftA3
               1 & (\_ ->
                kf $ A $
                 2 & (\_ ->
                  kf $ A $
                   kp 3))
F $ \kp kf -> kf (A $                 -- equivalent
               kf $ A $
                kf $ A $
                 kp 3)

kf A (kp 1)
kf A (1 & (\_ -> kf A kp 2))

-- fold
x :: F A Int
x = do
  liftA 1
  liftA 2
  liftA 3
runF x Pure Free = Free $ A $
                    Free $ A $
                     Free $ A $
                      Pure 3
```

==========
======